Jev engineering: the reference playbook

Author: Mr. Buzzoni (@Mr_Buzzoni), published 29 September 2026

Source: https://x.com/mr_buzzoni/status/2104783226833186920

About: TypeSafe AI's Jev, a "System One" decision model from Diogo Almeida's team that

answers typed questions with calibrated probabilities instead of generating text.

Retrieved: 8 November 2026, for the jev-harness project.

Third-party material reproduced for reference, with the promotional closing section kept

intact so the attribution stays honest. The harness in this repository implements the

architecture this document describes: code rules first, a decision model for bounded calls,

a chat model for the cases it is unsure about, and a human after that.

How to find the decisions hiding inside your agents, write questions a system one model answers well, route on calibrated confidence and hand the hard cases to Kimi K3. The API, the code, the weak spots and the math.__

Open any agent transcript and count the moments where the model wasn't creating anything. Which tool next. Is this source relevant.

Did the test pass. Is this command safe. Is the task done. In most loops those small calls outnumber the real work, and every one of them is billed as a full generation from a frontier model.

On September 15 TypeSafe AI came out of stealth with a model built only for those calls. Jev never writes a word.

You send it state and typed questions, it sends back answers with probabilities, usually in around 100 milliseconds, at $0.042 per million input tokens with output free.

On TypeSafe's own workflow evals it came out 193.6x faster and 444.6x cheaper than the LLMs it was measured against, and the company itself calls those numbers the high end of what to expect.

Jev engineering is the craft of using it well: finding the hidden decisions, writing questions the model can answer, routing on its confidence, and knowing where it breaks. This is the full playbook, with Kimi K3 as the model that takes the cases Jev isn't sure about.

1/ Why a Model That Can't Write Is Useful

Diogo Almeida co-authored InstructGPT, the RLHF work that turned GPT-3 into ChatGPT. His argument for Jev starts with a critique of his own technique: models trained to please human raters learn to look right, and their stated confidence stops meaning anything. If a model can do a task 95% of the time but can't tell you which 5% it's getting wrong, you can't automate that task.

Jev is trained with what TypeSafe calls RLCD, reinforcement learning for calibrated decisions. The target is honest probabilities: answers given 90% should be right about 90% of the time across your data.

__Individual answers can still be wrong, which is exactly why the confidence number matters. It tells your code which answers to act on and which to send somewhere else.__

A real run shows the idea. Hassan (@nutlope (https://x.com/nutlope)) screened 100 emails, half legitimate and half fraud. Jev classified all 100 in 1.42 seconds. The 31 answers under 95% confidence went to Kimi K3 for a full read. The pipeline finished 96 of 100 right in 16 seconds for about $0.07, and Jev's share of that bill was a third of a cent.

2/ Step One: Sort Every Step of Your Agent

Go through one real transcript and put every step in one of four buckets.

```
If the step...  It belongs to  Examples
Has to create text, code or a plan  An LLM  The draft, the fix, the summary, the reason a human will read
Follows an exact rule  Code  Retry limits, budgets, allowlists, date math, counting
Picks from answers you can list in advance  Jev  Route, rank, keep or drop, safe or not, done or not
Can't be undone  You  Send, pay, publish, delete, change permissions
```

The third bucket is always bigger than people expect, and it's the one Jev engineering moves. Everything else stays where it is.

3/ The Primitives and the API

Three question types cover the third bucket.

```
Type  Asks  Returns  Rules
Choice  Pick one of your options  choice, a probability for every option, confidence  Up to 255 options, probabilitie
Score  Place it on your rubric  score, which can land between levels, plus the distribution  2 to 10 ordered levels
Noul  Yes or no  noul, the probability of yes  0.5 means it can't tell, never "medium"
```

The facts you'll need while building:

- **Endpoint:** POST https://api.typesafe.ai/v1/systemone (https://api.typesafe.ai/v1/systemone) with state, model and questions. Python SDK: pip install typesafe-sdk, then client.system_one(...)
- **Model:** pin jev-1.13.0 once your thresholds are tuned. jev-latest moves.
- **Size:** state plus all questions share about 64K tokens, and state plus the longest single question must fit in about 32K (roughly 150,000 characters of English).
- **Input:** text only. Strings, JSON objects or arrays, no images.
- **Rate limits:** 1,200 requests and 250,000 tokens per second on jev-1.13, and TypeSafe says those will move as capacity lands.
- **Speed and price:** 70 to 500 ms end to end, $0.042 per million input tokens, output free. 10,000 decisions at 1,000 tokens each cost $0.42.

4/ Ask Everything in One Call

Questions in a single request are evaluated in parallel against the same state.

TypeSafe calls this speculative fan-out, and it's the single biggest cost lever you control. In one of their tests, 13 questions over a 53,777-character document ran 12.2x cheaper and 10x faster as one call than as thirteen, mostly because the state was sent once.

from typesafe_sdk import Choice, Noul, Score, TypeSafeClient

client = TypeSafeClient(model="jev-1.13.0")

r = client.system_one(

state={"ticket": ticket_text, "customer": {"plan": "pro", "refunds_90d": 0}},

questions={

"department": Choice(

instructions="Which team should own the problem described in `ticket`?",

criteria={

"billing": "Charges, invoices, plan changes",

"bug": "Something in the product does not work as documented",

"howto": "The customer asks how to do something that already works",

"other": "None of the above fits",

}),

"frustration": Score(

instructions="How frustrated is the customer, judged from `ticket` only?",

criteria=[

"Neutral or positive, no complaint about the experience",

"Annoyed but polite, describes wasted time",

"Angry, mentions cancelling, a public review or a chargeback",

]),

"refund_requested": Noul(

instructions="Does `ticket` ask for money back, a refund or a chargeback?"),

},

)

dept = r.answers["department"] # .choice .probabilities .confidence

mood = r.answers["frustration"].score # can land between levels, e.g. 1.3

refund = r.answers["refund_requested"].noul

Two boundaries apply. Questions in one call can't read each other's answers, so if a decision depends on new evidence, fetch it first and ask again. And fan-out only pays when the questions share the state; unrelated questions belong in separate calls

5/ Writing Questions Is the Actual Skill

Most bad Jev answers trace back to a bad question. These rules come from TypeSafe's docs and from people who shipped with it.

- **The question ID is never sent to the model.** Naming a field is_fraud tells Jev nothing. The meaning has to live in instructions and criteria.
- **Ask one judgment per question.** "Is this urgent and from a paying customer" is two questions. Split them and combine the answers in code.
- **Describe situations, not degrees.** "Mentions cancelling or a chargeback" is checkable. "Very frustrated" is a mood, and so are bare numbers as levels.
- **Write every level so it stands alone.** Levels are judged independently, so "worse than the previous level" means nothing to the model.
- **Phrase every Noul so a high number means yes.** A Noul where true means "not safe" is a bug waiting to happen.
- **Always add an exit.** other or none on every Choice, because the model has to pick something and an exit gives uncertainty a place to go.
- **Point at the exact field.** Backticked paths like `order.charges` tell Jev which part of the state a question is about.
- **Send evidence, and filter in code first.** Accuracy drops as irrelevant state piles up. Seven sources with their claims beat "the research looks mostly done," and a search result trimmed to the relevant rows beats the whole page.
- **Turn extraction into selection.** Jev can't produce a value that isn't in your schema. If you need to pull a vendor name from an invoice, give it the list of known vendors and ask it to pick one.

6/ Know Where It Breaks

TypeSafe publishes a "jaggedness" page for jev-1.13. Every item on it has a harness fix.

```
Weak spot  What happens  What the harness does
Reads questions literally  A loosely worded question gets a literal answer  Treat questions like code: version, test,
Not a calculator  Counting errors grow with size  Count, sum and compare in code, pass the result in
Dates are text  "After March 3" isn't an ordered comparison  Compute date logic in code
Context rot  Irrelevant state lowers accuracy  Trim state to what the questions need
Trusts the state  Adversarial text inside the state can shift answers  Deterministic checks and permissions stay in c
Text in, numbers out  It can't explain itself or read a screenshot  An LLM handles anything that needs words or visio
Multi-hop reasoning  Chains of inference degrade  Break the chain into several calls with state updates
```

The last line of defense is the same for all of them: type safety means Jev can't return a value outside your schema, but it can still return the wrong valid value. Confidence thresholds and a second model are how you catch those.

7/ The Cascade: Jev First, Kimi K3 on the Hard Cases

Put it together and every decision falls through four layers, stopping at the first one that's sure.

```
Layer  What it handles  Speed  Cost
Code  Exact rules  Instant  Free
Jev  Bounded calls above your threshold  ~100 ms  $0.042 per million input tokens
Kimi K3  Answers under the threshold, and anything that needs a written reason  Seconds  $3 in / $15 out per million
You  Irreversible actions, and cases both models are unsure about  Whenever you look  Your time
```

Kimi K3 fits the second slot for practical reasons. An escalated case often needs the whole thread and history, and K3's 1M-token context is priced the same at any length. It reads the screenshots and scanned PDFs that Jev can't. Its weights are open, so the same code runs against Moonshot's API, Together (Hassan's setup) or your own host.

import json, os

from openai import OpenAI

from typesafe_sdk import Choice, TypeSafeClient

CRITERIA = {

"customer": "A current customer asking about an order, a bill or their account",

"lead": "Someone asking about prices, a demo or working together",

"vendor": "Someone selling a product or service to us",

"fraud": "Urgent payment request, changed bank details, spoofed sender or a login link",

"other": "None of the above fits",

}

THRESHOLD = 0.95 # Hassan's setting for fraud, tune it per decision

jev = TypeSafeClient(model="jev-1.13.0")

kimi = OpenAI(api_key=os.environ["KIMI_API_KEY"],

base_url=os.environ.get("KIMI_BASE_URL", "https://api.moonshot.ai/v1"))

def ask_kimi(email, jev_guess):

reply = kimi.chat.completions.create(

model="kimi-k3",

response_format={"type": "json_object"},

messages=[{"role": "user", "content":

f"Classify this email as one of {list(CRITERIA)}. "

f"A fast classifier guessed '{jev_guess}' with low confidence. "

'Answer as JSON: {"category": "...", "reason": "...", "sure": true}\n\n' + email}],

)

return json.loads(reply.choices[0].message.content)

def route(email):

r = jev.system_one(

state={"email": email},

questions={"category": Choice(

instructions="What does the sender of `email` want?", criteria=CRITERIA)},

)

a = r.answers["category"]

if a.confidence >= THRESHOLD and a.choice != "other":

return {"category": a.choice, "by": "jev", "confidence": a.confidence}

k = ask_kimi(email, a.choice)

if k.get("sure"):

return {"category": k["category"], "by": "kimi-k3", "reason": k["reason"]}

return {"category": "review", "by": "you", "jev": a.choice, "kimi": k["category"]}

other never routes on its own. K3 gets Jev's guess as context. And when both are unsure, you get both answers side by side, which is the fastest review there is.

8/ Plug It Into the Harness You Already Have

You don't need a new framework. Jev already ships inside the three most common ones.

LangChain (langchain-typesafe) has two middlewares. ModelRouterMiddleware lets Jev pick the cheapest model that can handle a request, against criteria you write.

AutoModeMiddleware checks every tool call for risk before it executes, the kind of dangerous-action classifier coding harnesses have kept closed-source until now:

from langchain.agents import create_agent

from langchain_typesafe.experimental.middleware import AutoModeMiddleware

agent = create_agent(YOUR_MODEL, middleware=[AutoModeMiddleware(tools=["bash"])])

Pydantic AI (pydantic-ai-slim[typesafe]) turns your output type into questions: Agent("typesafe:jev-latest", output_type=Triage).

A bool becomes a yes or no, a Literal becomes a Choice, an IntEnum with a docstring on each member becomes a Score, and X | None adds a "None of these" option. Its FallbackModel pattern hands unsure answers to an LLM, which is the cascade above in two lines.

Vercel AI SDK exposes Jev through experimental_evaluate with the @ai (https://x.com/ai)-sdk/typesafe-ai provider, and through the AI Gateway as typesafe-ai/jev at the same price.

9/ What People Are Already Running

```
Build  What Jev decides  Result
Browser Use jev-ultrafast  The next action and which element to act on, from a fresh list each step  Zürich to London
Fraud screen (Hassan)  Legit or fraud, with the unsure 31 sent to Kimi K3  96/100 correct, 16 s, about $0.07
1kpapers.com  Which of 24 topics each paper belongs to  1,018 papers for $0.08 at 256 ms median, against $3.99 to sum
```

```
Inbox triage (Riley Brown)  What each email needs  500 emails for 3.5 cents
Computer use (awlevin)  Each step of a desktop task  $0.0002 per decision against $0.032 for Opus 5, 0.13 to 0.38 s a
Instant compaction (tamara)  Which tool calls in a session are still relevant  A Claude session cut from nearly 1M to
jev-trader (Jarrod Watts)  One trading decision per block  About 81 ms model latency
```

The pattern repeats: the menu of options is rebuilt from live state at every step, Jev picks, and code or a small model does the rest.

10/ Jev Steering a Kimi K3 Swarm

The same idea scales up. When the job is big, Kimi K3's Agent Swarm does the building with up to 300 sub-agents, and Jev handles the between-turn calls: which role runs next, which returns to keep, and whether the goal is met.

Three Jev calls per turn cost a fraction of a cent, so the loop can check "are we done" after every turn and stop itself. When the done-check lands in the middle band, one K3 call with the full state decides.

11/ Thresholds and Rollout

Set the threshold per action, not per model. TypeSafe's docs use 0.5 as a floor for human review and 0.9 before anything destructive, and call them examples rather than defaults. Filing a newsletter can go automatic much lower than flagging fraud.

Run a shadow week. Let Jev label while the old path decides. Then compare by confidence band: if the 0.9+ answers match yours nearly every time, that band goes automatic first.

Pin the version and log everything. Model ID, full distribution, threshold, which layer answered, what happened next. The log becomes your evaluation set, and a pinned version means your thresholds still mean the same thing next month.

Treat questions like code. A change to criteria changes production behavior. Version it, test it on the labeled set, roll it back when it drifts.

12/ The Math

10,000 emails a month. Assumptions: about 1,000 input tokens per email for Jev, 1,500 in and 300 out for a full model call, 10% escalated.

```
Setup  Price  Monthly cost
A frontier model reads every email  $10 in / $50 out per million  $300
Kimi K3 reads every email  $3 in / $15 out per million  $90
Jev on everything, Kimi K3 on the 10% it's unsure about  Jev $0.42 + K3 $9.00  $9.42
```

TypeSafe's own cascade estimate lands in the same place at scale: about $6,480 instead of $30,400 for a million support tickets.

13/ What It Pays

```
Channel  What it pays  What you need first
Triage setups for small businesses  A setup fee plus a monthly retainer for the cascade, the review queue and a thres
Fraud and invoice screening for finance teams  A fee per mailbox; one stopped payment to a spoofed vendor covers year
Agent cost audits  A fixed fee to map a team's agent loop and move its decision calls to Jev  The four-bucket sort fr
```

The third one is the fastest to sell. Every team running agents is paying frontier prices for decisions, and the audit shows them the exact calls to move.

The Short Version

Sort every step of your agent into create, exact rule, pick from known answers, and irreversible. Move the third bucket to Jev, ask every related question in one call, write criteria as situations with an exit option, and route on confidence per action. Send what Jev isn't sure about to Kimi K3, and keep the irreversible steps for yourself.

Start with one decision your agent makes fifty times a day.

The stack:

■ Jev by TypeSafe AI

■ https://typesafe.ai (https://typesafe.ai/)

■ Jev API docs

■ https://docs.typesafe.ai/api (https://docs.typesafe.ai/api)

■ Kimi K3 API

■ https://platform.kimi.ai (https://platform.kimi.ai/)

■ Hassan's Jev + Kimi K3 fraud pipeline

■ https://github.com/Nutlope/jev-fraud (https://github.com/Nutlope/jev-fraud)

And if you found this useful:

- Bookmark this article. The links change and new repos pop up weekly, you'll need this as a reference
- For weekly deep dives into AI architecture, quant trading, and the agent economy, follow me:@polydao (https://x.com/@polydao)
- - here I share my raw prompts, custom skills, and alpha that's too early for XJoin the TG Channel:Buzzoni Notes (https://t.me/+Wf8q84QkpyJhNjIy)

• building a second brain with ■laude & Obsidian • • @cele8ex (https://x.com/cele8ex) member • co-founder @polynternet (https://x.com/polynternet) •